```
# ============================================================
# EMAIL SPAM DETECTION USING KNN AND SVM
# ============================================================

# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# ============================================================
# 2. LOAD DATASET
# ============================================================
import pandas as pd
from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')
# Read dataset
df = pd.read_csv('/content/drive/MyDrive/emails.csv')
# Display first 5 rows
print("First 5 rows:")
print(df.head())
# Display dataset shape
print("\nDataset shape:")
print(df.shape)
# Display column names
```

```
print("\nColumn names:")
print(df.columns)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# ============================================================
# 3. CHECK MISSING VALUES
# ============================================================

print("\nMissing values:")
print(df.isnull().sum())

# ============================================================
# 4. REMOVE UNNECESSARY COLUMN
# ============================================================

# The Kaggle dataset commonly contains an unnecessary
# unnamed/index column.

if "Unnamed: 0" in df.columns:
    df.drop("Unnamed: 0", axis=1, inplace=True)

# ============================================================
# 5. IDENTIFY TEXT AND TARGET COLUMNS
# ============================================================
```

```
print("\nColumns after cleaning:")
print(df.columns)

# For this dataset:
# features are word counts (all columns except 'Email No.' and 'Prediction')
# target column = Prediction

X = df.drop(columns=['Email No.', 'Prediction'])
y = df["Prediction"]

# ================================================================
# 6. HANDLE MISSING VALUES
# ================================================================

X = X.fillna(0) # Assuming 0 is a reasonable fill value for word counts

# Remove rows where target is missing
valid_rows = y.notna()

X = X[valid_rows]
y = y[valid_rows]

# ================================================================
# 7. CHECK CLASS DISTRIBUTION
# ================================================================

print("\nClass distribution:")
print(y.value_counts())

print("\nClass distribution in percentage:")
print(y.value_counts(normalize=True) * 100)

# ================================================================
# 8. CONVERT TEXT INTO NUMERICAL FEATURES
#    USING TF-IDF
# ================================================================

print("\nFeature matrix shape:")
print(X.shape)

# ================================================================
# 9. SPLIT DATA INTO TRAINING AND TESTING
```

```
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)

# ============================================================
# 10. K-NEAREST NEIGHBORS (KNN)
# ============================================================

knn = KNeighborsClassifier(
    n_neighbors=5,
    metric="cosine"
)

knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

# ============================================================
# 11. SUPPORT VECTOR MACHINE (SVM)
# ============================================================

svm = SVC(
    kernel="linear",
    random_state=42
)

svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)

# ============================================================
# 12. EVALUATE KNN
# ============================================================
```

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)

precision_knn = precision_score(y_test, y_pred_knn, average='weighted', zero_division=0) # Added average and zero_division for multi-class

recall_knn = recall_score(y_test, y_pred_knn, average='weighted', zero_division=0) # Added average and zero_division for multi-class

f1_knn = f1_score(y_test, y_pred_knn, average='weighted', zero_division=0) # Added average and zero_division for multi-class

print("\n==================================")
print("KNN PERFORMANCE")
print("==================================")

print("Accuracy  :", accuracy_knn)
print("Precision :", precision_knn)
print("Recall    :", recall_knn)
print("F1 Score  :", f1_knn)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_knn, zero_division=0)) # Added zero_division

# ============================================================
# 13. CONFUSION MATRIX - KNN
# ============================================================

cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")

plt.show()
```

```
# ============================================================
# 14. EVALUATE SVM
# ============================================================

accuracy_svm = accuracy_score(y_test, y_pred_svm)

precision_svm = precision_score(y_test, y_pred_svm, average='weighted', zero_division=0)
# Added average and zero_division for multi-class

recall_svm = recall_score(y_test, y_pred_svm, average='weighted', zero_division=0) #
Added average and zero_division for multi-class

f1_svm = f1_score(y_test, y_pred_svm, average='weighted', zero_division=0) # Added
average and zero_division for multi-class

print("\n==================================")
print("SVM PERFORMANCE")
print("==================================")

print("Accuracy  :", accuracy_svm)
print("Precision :", precision_svm)
print("Recall    :", recall_svm)
print("F1 Score  :", f1_svm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm, zero_division=0)) # Added zero_division

# ============================================================
# 15. CONFUSION MATRIX - SVM
# ============================================================

cm_svm = confusion_matrix(y_test, y_pred_svm)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Blues"
)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix")

plt.show()

# ============================================================
# 16. COMPARE KNN AND SVM
# ============================================================

results = pd.DataFrame({

    "Model": [
        "KNN",
        "SVM"
    ],

    "Accuracy": [
        accuracy_knn,
        accuracy_svm
    ],

    "Precision": [
        precision_knn,
        precision_svm
    ],

    "Recall": [
        recall_knn,
        recall_svm
    ],

    "F1 Score": [
        f1_knn,
        f1_svm
    ]
})

print("\n==================================")
print("MODEL COMPARISON")
print("==================================")

print(results)
```

```
# ============================================================
# 17. PERFORMANCE COMPARISON GRAPH
# ============================================================

results.set_index("Model").plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title("KNN vs SVM Performance")
plt.ylabel("Score")
plt.ylim(0, 1)

plt.xticks(rotation=0)

plt.legend()
plt.show()
```